

MEAM 5200-Lab2: Inverse Kinematics

Hongbo Nie, Muwen Liu

June 4, 2026

1 Introduction

Robotic manipulators require relating joint variables to the position and orientation of the end effector. This lab focuses on velocity kinematics and inverse kinematics (IK) for the 7-degree of freedom Franka Panda robot. Velocity kinematics uses the Jacobian to relate joint velocities to the end effector velocity vector, while Inverse Kinematics determines the joint variables required to achieve a desired end effector pose. In this lab, Jacobian-based velocity kinematics inverse velocity kinematics, and a numerical inverse position kinematics solver using the pseudo-inverse are implemented. Trajectory tracking for line and figure-eight paths is also implemented, and performance is evaluated in simulation and hardware.

2 Method

Code from both Hongbo and Muwen were used in this lab.

2.1 Jacobian

The function is implemented in file `calcJacobian`. The Jacobian describes the relationship between joint velocities and the linear and angular velocities of the end effector:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = J(q)\dot{q} \quad (1)$$

where q are the joint variables, v is the end effector's linear velocity, ω is the end effector's angular velocity. In the implementation, the Jacobian is constructed using the joint axes and positions obtained from forward kinematics. Since Franka Panda robot's joints are all revolute joints, therefore, the linear velocity jacobian is $z_{i-1} \times (o_n - o_{i-1})$ and the angular velocity jacobian is z_{i-1} . The Manipulator Jacobian is given by:

$$J_i(q) = \begin{bmatrix} J_{v_i} \\ J_{\omega_i} \end{bmatrix} = \begin{bmatrix} z_{i-1} \times (o_n - o_{i-1}) \\ z_{i-1} \end{bmatrix}$$

where z_{i-1} is the axis of joint $i - 1$, o_e is the end effector origin position, and o_{i-1} is the origin position of the joint $i - 1$. Stacking all columns create a 6×7 Jacobian, with the top three rows corresponding to linear velocity and the bottom three rows corresponding to angular velocity.

$$J(q) = [J_1 \quad J_2 \quad J_3 \quad J_4 \quad J_5 \quad J_6 \quad J_7]$$

2.2 Inverse velocity kinematics

Inverse velocity kinematics function implemented in `IK_velocity` determines the joint velocities required to achieve a desired end effector velocity. From equation (1), it can be seen that the desired joint velocities can be obtained by taking the inverse of the Jacobian multiply with the end

effector velocity vector. However, since the size of the Jacobian is 6×7 for Franka Panda arm, therefore, the pseudo inverse is utilized for obtaining the joint velocities:

$$\dot{q} = J^\dagger \begin{bmatrix} v \\ \omega \end{bmatrix}$$

This corresponding to solving the least-squares problem:

$$\dot{q} = \underset{q}{\operatorname{argmin}} ||J(q)\dot{q} - \begin{bmatrix} v \\ \omega \end{bmatrix} ||^2$$

In the implementation, rows corresponding to unconstrained directions (specified as np.nan) are removed before solving which leave only the constraint components that will affect the solution to be solved.

2.3 Inverse kinematics

The inverse kinematics solver and helper functions are implemented in `IK_position_null.py`. The helper function `displacement_and_axis(target, current)` returns the translational error and rotational error vector which help correcting the end effector motion orientation. The translational error is calculated as:

$$e_p = p_{target} - p_{current}$$

which points from the current end effector position to the desired one. The orientation error is computed using the relative rotation:

$$R_{rel} = R_{curr}^T R_{des}$$

the skew-symmetric part of R_{rel} which stored as a is used to extract a vector $e_R = R_{curr}a$ representing the axis of rotation which serves as the orientation error. The combined error is written as:

$$e = \begin{bmatrix} e_p \\ e_R \end{bmatrix}$$

which the primary task is to reduce this pose error.

Helper function `distance_and_angle(G,H)` computes the Euclidean distance and rotation angle between two poses (G and H)

$$d = ||p_G - p_H||$$

$$\theta = \cos^{-1}\left(\frac{\operatorname{trace}(R_{rel}) - 1}{2}\right)$$

These differences serve to evaluate whether the solution satisfies the required tolerances.

The main inverse kinematics solver is implemented in the function `inverse(target,seed,alpha)`. Starting from an initial guess q_0 , the joint variables are updated iteratively:

$$q_{k+1} = q_k + \alpha dq$$

at each iteration, the current end effector pose is obtained using forward kinematics, and the pose error e is computed. The joint update for the primary task is obtained by solving a least-squares problem using the jacobian:

$$dq_{primary} = J^\dagger e$$

which minimizes the error between the desired and achieved end-effector motion. Because the Franka Panda robot has 7 DOF (redundant), a secondary task is introduced to keep the joint variables close to the center of allowable ranges:

$$dq_{secondary} = -k(q - q_{center})$$

and to ensure that the secondary task does not interfere with the primary objective, it is projected into the null space of the Jacobian:

$$dq = dq_{primary} + (I - J^\dagger J) dq_{secondary}$$

The iteration continues until either the maximum number of iterations is reached or when the update is inconsiderable: $\|dq\| < \epsilon$.

2.4 Trajectory tracking

Two trajectories are implemented: a line trajectory and a figure-eight trajectory, both defined as functions of time. The line trajectory is defined as :

$$x_{des}(t) = x_0 + \begin{bmatrix} 0 \\ L \sin(ft) \\ 0 \end{bmatrix}$$

$$v_{des}(t) = \begin{bmatrix} 0 \\ L f \cos(ft) \\ 0 \end{bmatrix}$$

The figure-eight trajectory is defined as:

$$x_{des}(t) = x_0 + \begin{bmatrix} r_x \sin(f_x t) \\ r_y \sin(f_y t) \\ 0 \end{bmatrix}$$

Since the motion in the y-direction oscillates at twice the frequency of the motion in the x-direction, the end effector traces a figure-eight shaped path in xy-plane. The corresponding linear velocity is:

$$v_{des}(t) = \begin{bmatrix} r_x f_x \cos(f_x t) \\ r_y f_y \cos(f_y t) \\ 0 \end{bmatrix}$$

For orientation, a time-varying orientation is defined using a rotation about the x-axis:

$$\theta(t) = -\pi + \frac{\pi}{4} \sin(ft)$$

with corresponding angular velocity:

$$\omega_{des}(t) = \begin{bmatrix} \frac{\pi}{4} f \cos(ft) \\ 0 \\ 0 \end{bmatrix}$$

The commanded linear and angular velocities are computed using a combination of feedforward and proportional feedback:

$$v = v_{des} + k_p(x_{des} - x)$$

$$\omega = \omega_{des} + k_r e_R$$

The desired spatial velocity is then converted into joint velocities using inverse velocity kinematics. Finally, the joint variables are updating iteratively:

$$q_{k+1} = q_k + \Delta t dq$$

This process allowing the robot to continuously track the desired trajectory in both position and orientation.

3 Evaluation

3.1 Eight trajectory implementation evaluation

The eight figure trajectory is tested in both simulation and the hardware. In simulation, the end effector initially follows the desired trajectory, however, over time, the configuration of the robot gradually drifted away from its initial configuration and cause the arm to move into an unnatural configuration as shown in figure 1. Which suggest a code/algorithm issue in the `follow.py`, potentially numerical error or lack of joint constraint handling. Because the issue was not detected prior to the lab, the same code was used for the hardware testing. As a result, while the robot was able to complete the first figure-8 cycle and trace the trajectory reasonably, failures began to occur during the second cycle, which one or more joints of the robot appeared to lock, preventing the end effector from continuing the motion and repeating the trajectory. This behavior matches with the phenomenon found in the simulation and suggest an issue in the control algorithm rather than hardware malfunction.

The phenomenon is likely due to the use of Jacobian-based velocity inverse kinematics control without additional without appropriate joint limit handling, consequently causing drift overtime. If orientation control is added, then the problem would become more challenging as the controller algorithm need to handle 6 parameters (3 position + 3 orientation) instead of only 3 position parameters. And since Franka Panda arm has 7 DOF, it only has $7 - 6 = 1$ extra DOF for handling additional constraints. This makes the system more likely to instability and infeasible configurations during trajectory tracking.

The video of the hardware execution can be accessed through the link below:

8 trajectory hardware testing video link: https://drive.google.com/file/d/1seeemJ80jadzWE0ck3-8bEyU-RE2/view?usp=drive_link

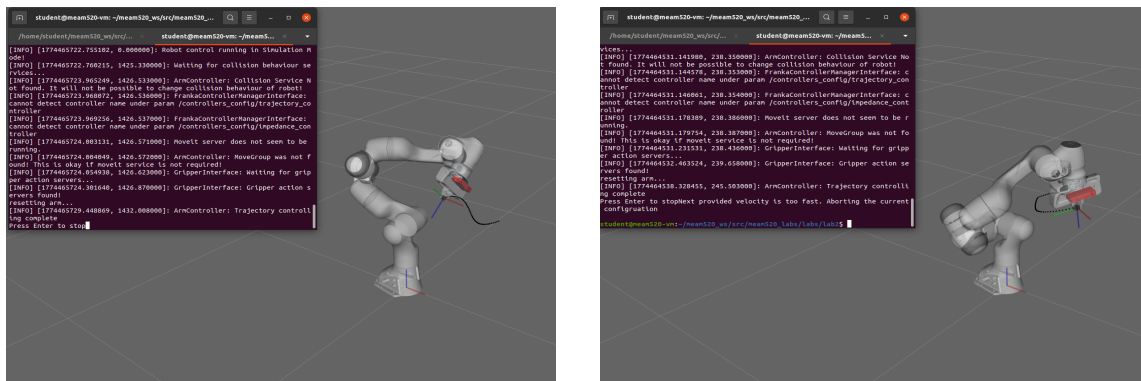


Figure 1: 8 trajectory simulation: Initial run (left) and Overtime config (right).

3.2 Evaluation of the implementation of position IK

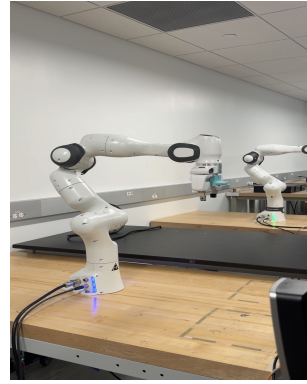
We tested five target poses. The corresponding simulation results and hardware configurations are shown below.

```

Target 0 located at:
[[ 1.  0. -0.  0.35]
 [ 0. -1.  0.  0.15]
 [-0. -0. -1.  0.45]
 [ 0.  0.  0.  1.  ]]
Solving...
Solution found in 0.04 seconds (14 iterations)
[INFO] [1774017740.522419]: ArmControl

```

(a) Simulation



(b) Hardware

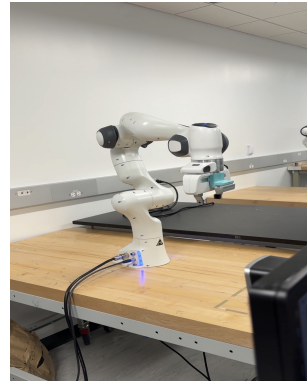
Figure 2: Target Pose 0

```

Target 1 located at:
[[ 1.  0. -0.  0.4]
 [ 0. -1.  0. -0.2]
 [-0. -0. -1.  0.4]
 [ 0.  0.  0.  1.  ]]
Solving...

```

(a) Simulation



(b) Hardware

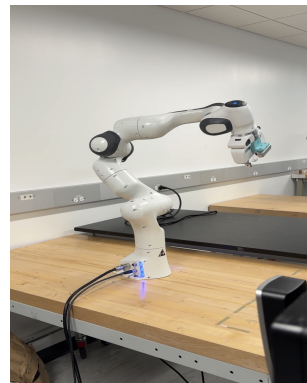
Figure 3: Target Pose 1

```

Press Enter to move to next target...
Target 2 located at:
[[ 1.  0. -0.  0.3  ]
 [ 0. -0.8660254  0.5  0.25 ]
 [-0. -0.5 -0.8660254  0.55 ]
 [ 0.  0.  0.  1.  ]]
Solving...
Solution found in 0.07 seconds (14 iterations).
[INFO] [1774017758.450278, 100.963000]: ArmControl

```

(a) Simulation



(b) Hardware

Figure 4: Target Pose 2

```

Press Enter to move to next target...
Target 3 located at:
[[ 0.8660254  0.      -0.5      0.45    ]
 [ 0.        -1.       0.       0.      ]
 [-0.5       -0.      -0.8660254  0.35    ]
 [ 0.         0.       0.       1.      ]]
Solving...
Solution found in 0.07 seconds (13 iterations).
[TIME] [1774017763.337575, 105.808000]: ArmCont

```

(a) Simulation



(b) Hardware

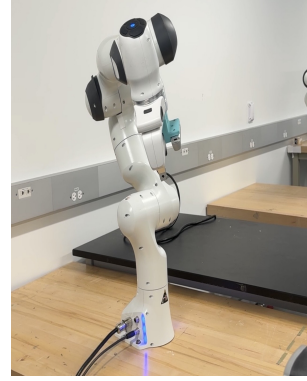
Figure 5: Target Pose 3

```

Press Enter to move to next target...
Target 4 located at:
[[ 0.8660254  0.5      -0.       0.25    ]
 [ 0.46193977 -0.80010315  0.38268343 -0.25    ]
 [ 0.19134172 -0.33141357 -0.92387953  0.5     ]
 [ 0.         0.       0.       1.      ]]
Solving...
Solution found in 0.07 seconds (14 iterations).
[TIME] [1774017763.337575, 110.649000]: ArmControll

```

(a) Simulation



(b) Hardware

Figure 6: Target Pose 4

We evaluated the numerical IK solver on five target poses in `visualize.py`. For each pose, we recorded the convergence time, number of iterations, and whether the solver successfully returned a valid solution.

Table 1: Numerical IK performance on new target poses

Target Pose	Success	Time (s)	Iterations	Simulation	Hardware
Pose 0	True	0.04	13	Yes	Yes
Pose 1	True	0.07	14	Yes	Yes
Pose 2	True	0.07	14	Yes	Yes
Pose 3	True	0.07	13	Yes	Yes
Pose 4	True	0.07	14	Yes	Yes

Table 2: Summary statistics of IK solver performance

Metric	Value
Mean time (s)	0.064
Median time (s)	0.07
Maximum time (s)	0.07
Mean iterations	13.6
Median iterations	14
Maximum iterations	14
Success rate	100%

The solver succeeded on all five target poses, corresponding to a 100% success rate. The elapsed time remained low and consistent, with a mean of 0.064 s and a maximum of 0.07 s. Similarly, the iteration count was stable, with a mean of 13.6 and a maximum of 14. These results indicate that the solver is both efficient and robust across the tested poses.

4 Analysis

Overall, the results make sense. The forward kinematics produced reasonable end-effector poses, and the inverse kinematics was generally able to find joint configurations that achieved the desired Cartesian poses.

The main surprising result was the difference between simulation and hardware performance. In simulation, the drift only became visible after several repetitions of the trajectory, whereas on the real Panda robot the drift appeared almost immediately. This difference suggests that the dominant sources of error are related to numerical accumulation and real-world physical effects that are not fully captured in simulation.

One major cause of drift is the accumulation of motion in the Jacobian null space. The commanded joint velocity is computed as

$$\dot{q} = \dot{q}_{primary} + (I - J^\dagger J)\dot{q}_{secondary} \quad (2)$$

The secondary objective is intended to improve joint posture or avoid singular configurations without affecting the primary end-effector task. However, in practice the secondary task does not stop acting even when the primary task is satisfied. Over time, this continuous null-space motion gradually changes the robot configuration, pushing the arm toward different joint positions. Although the end-effector may still approximately follow the desired trajectory, the internal posture of the robot drifts, which can lead to poorer manipulability or configurations closer to joint limits.

Another important factor is discrete-time integration error. The controller updates joint angles using

$$q_{k+1} = q_k + \Delta t \dot{q} \quad (3)$$

Even small biases in the computed joint velocity accumulate over time due to this integration process. This effect becomes especially noticeable for periodic trajectories such as straight-line or figure-eight motions, where small errors at each timestep compound and gradually produce visible deviation from the desired path.

The difference between simulation and hardware behavior can be explained by the presence of real-world effects that are not fully modeled in simulation. Simulation assumes perfect actuation and an ideal kinematic model, so errors mainly come from numerical approximation and are relatively small. On the real Panda robot, however, additional factors such as gravity, joint friction, actuator tracking error, and calibration inaccuracies introduce persistent bias in the executed motion. Because velocity control integrates joint velocities over time, even small systematic errors can accumulate quickly, causing the end effector to drift downward. This explains why drift appears much earlier in hardware experiments than in simulation.

Based on these observations, the Panda robot performed best when following simple, straight-line trajectories. These motions are less sensitive to small Jacobian errors and don't require large joint reorientations, so the system is stable and accurate. However, the robot performs less well for motions that require maintaining precise position over long durations, repeated periodic trajectories, or configurations where the Jacobian becomes poorly conditioned. In these situations, accumulated numerical error and null-space motion can gradually degrade tracking accuracy and cause configuration drift.

Velocity control is useful when we want smooth and continuous motion, such as tracking a trajectory in Cartesian space. It allows the robot to move naturally between poses and makes it easier to incorporate secondary objectives like avoiding joint limits or maintaining good posture through null-space motion. Position control is more suitable when we need the robot to reach and hold a precise pose. Because position error is continuously corrected, position control helps prevent drift and is more stable for tasks like grasping or maintaining a fixed end-effector pose. In general, velocity control is better for motion generation, while position control is better for accuracy and stability.

The inverse velocity kinematics doesn't always give us the same solution because the Panda robot has redundant degrees of freedom. For inverse velocity kinematics, the Jacobian pseudo-inverse gives one valid solution, but different joint velocities can produce the same end-effector motion due to null-space freedom. The result can vary depending on the secondary task or current configuration. Inverse position kinematics also does not always give the same solution. Multiple joint configurations can correspond to the same end-effector pose, and iterative solvers may converge to different solutions depending on the initial guess.

Failure to find a solution does not always mean the pose is unreachable or that the implementation is incorrect. Numerical IK methods can fail due to poor initial guesses, joint limits, or singular configurations where the Jacobian becomes poorly conditioned. In some cases, the solver may not converge even though a valid solution exists. Some poses may also lie near the edge of the workspace or require uncomfortable joint configurations, which makes convergence harder. Therefore, solver failure is often due to numerical sensitivity or physical constraints rather than coding errors.